

Lab exercises

Part 02 —  Deep Learning · 02-deep-learning/lab/EXERCISES.md

Lab — Deep Learning

1. Prove the nonlinearity matters (45 min)

In `../code/backprop_from_scratch.py`, the XOR demo shows a linear model failing.

- **1a.** Build a 3-layer network with `activation` replaced by the identity function. Train it on XOR.
- **1b.** Show numerically that its predictions match a single-layer model's.
- **1c.** Write the one-line algebraic reason.

Deliverable: the accuracy numbers and your one-line proof.

2. Add a layer type to the from-scratch MLP (2h)

Extend `MLP` with **dropout** and **L2 weight decay**, both implemented by hand.

Checks: - Inverted dropout: at train time, divide surviving activations by `(1-p)` so the expected value is unchanged. Verify empirically that `E[a_train] ≈ a_eval` over 10,000 samples. - With `p=0.5` on the digits dataset, the train/val gap should *narrow* versus `p=0.0`. - Your gradient check still passes (disable dropout during the check — it's stochastic).

3. Vanishing gradients, measured (1.5h)

Build a 12-layer MLP. For sigmoid, tanh, and ReLU activations, print the gradient norm of **each layer** after one backward pass on random data.

Deliverable: a 3×12 table of gradient norms. **Checks:** - With sigmoid, layer-1's gradient norm is orders of magnitude below layer-12's. - Add residual connections (`h = h + f(h)`) and show the early-layer gradients recover. - Answer: how many layers of sigmoid before the layer-1 gradient falls under $1e-8$?

4. The debugging gauntlet (2h)

Take `../code/pytorch_training_loop.py` and introduce these five bugs **one at a time**. For each, record the *observable symptom* before you look at the code.

1. Delete `optimizer.zero_grad()`.
2. Delete `model.eval()` in `evaluate()`.

3. Apply `softmax` to the model output before `CrossEntropyLoss`.
4. Set `lr=50`.
5. Shuffle the labels of the validation set only.

Deliverable: a 5-row table: bug → symptom → how fast the 15-minute checklist would have caught it. This is the most useful exercise in the module. Do not skip it.

5. Build a CNN and beat an MLP (3h)

On CIFAR-10 (or Fashion-MNIST if you're on CPU):

- **5a.** Train an MLP baseline. Record test accuracy and parameter count.
- **5b.** Train a small CNN with a similar parameter count.
- **5c.** Add data augmentation (random crop + horizontal flip). Record the delta.
- **5d.** Fine-tune a pretrained ResNet-18 with a replaced head.

Checks: CNN > MLP at comparable parameter counts. Augmentation adds ≥ 2 points. Fine-tuning beats everything, on far fewer epochs. Explain in two sentences why 5d wins.

6. Schedules and the LR-range test (2h)

Implement an LR-range test: start at $1e-7$, multiply the LR by 1.1 every batch, plot loss vs. LR on a log axis.

Checks: - The curve is flat, then descends, then explodes. Your chosen LR is roughly **one order of magnitude below the explosion point**. - Compare constant LR vs. warmup+cosine at that LR over a fixed epoch budget. Report both final accuracies.

7. Mixed precision and throughput (1.5h, needs a GPU)

Measure samples/second and peak memory for: fp32, bf16 autocast, bf16 + `torch.compile`, and bf16 + gradient checkpointing.

Deliverable: a 4-row table with throughput, peak memory, and final accuracy. **Check:** accuracy should be within noise across all four. If bf16 changed your accuracy meaningfully, say why that's suspicious.

8. Capstone — a reproducible training project (4h)

Produce a small repo containing: - `config.yaml` with every hyperparameter (nothing hard-coded), - seeded, deterministic runs, - checkpointing of the best model plus optimizer/scheduler state, - a logged run (MLflow, W&B, or a CSV + plots), - a `RESULTS.md` with the LR sweep, the ablation table, and the failure modes you saw.

Check: a colleague clones it, runs one command, and reproduces your headline number to within the seed noise you documented. That `RESULTS.md` failure list is the seed of your eval suite in Module 15 [15-ai-evals].